

Programming

William Schultz

June 15, 2026

1 Concurrency

When trying to program in a concurrent setting with multiple threads, *mutexes* and *condition variables* are often seen as the two core, fundamental concurrency primitives needed to safely manage and coordinate shared state between threads.

In C++, for example, we have `std::mutex` and `std::condition_variable` classes that provide a way to manage and coordinate shared state between threads. A mutex is simply a lock that can be acquired and released by a thread, and can only be held atomically by one thread at a time. A condition variable is always associated with some mutex, and in general there can be multiple condition variables (each corresponding to different waiting predicates) associated with the same mutex. This provides a fundamental way to ensure (1) safe concurrent access to shared state and (2) an efficient way for threads to correctly wait on some condition to become true of the concurrent object before proceeding.

For example, if you want to implement a *bounded concurrent queue* from scratch in a low level language like C++, you start with a basic queue object and protect its accesses by a mutex. This is fine for safe concurrent access, but what happens when a thread wants to push onto a queue that is already full, or pop from an empty queue? You can do this naively by *spin waiting* i.e. simply drop the mutex temporarily (to give others a chance to make progress on the queue), and then re-acquire it and check for the enabling condition again. This is inefficient, though, since it leads to constant wasted CPU cycles for all threads waiting on this condition.

We want a more efficient way to handle this by allowing threads to “sleep” when a condition is not yet held, and be woken up when the condition check becomes true. In essence, a condition variable is really just a queue of waiting threads associated with (1) some mutex and (2) a common predicate/condition they are waiting on. When the condition variable is “signaled”, it goes to either one or all of the threads on this queue and wakes them up, so they can re-acquire the mutex and check their condition before proceeding.

References